Glossário Detalhado e Enciclopédia Conceitual de DevSecOps

Projeto Final DevSecOps Clube Aurora (Turma 1746)

> **Guia de Consulta Rápida e Aprofundamento:**
> Este glossário foi especialmente estruturado para apoiar os alunos de todas as trilhas em especial da **Trilha Nível 1 (Teórica Guiada)** e **Nível 2 (Mista)** , detalhando minuciosamente os conceitos teóricos, termos de governança, siglas de mercado e vulnerabilidades presentes no caso prático do **Clube Aurora**.

Índice Temático

1. Metodologias de Segurança e Cultura Ágil
2. A Tríade CIA no Contexto do Negócio
3. Modelagem de Ameaças: Framework STRIDE Esmiuçado
4. Governança, Identidade e Controle de Acesso (IAM/RBAC)
5. Tipos de Testes Automatizados na Esteira DevSecOps
6. Anatomia das Vulnerabilidades do Clube Aurora (F1 a F7)
7. Hardening e Segurança em Contêineres (Docker)
8. Auditoria, Observabilidade e Não-Repúdio
9. Gestão de Riscos, Exceções e Métricas Corporativas

---

1. Metodologias de Segurança e Cultura Ágil

Shift-Left Security ("Mover a Segurança para a Esquerda")

- **O que é:** No ciclo tradicional de desenvolvimento (Waterfall), o software era desenvolvido, testado funcionalmente e, apenas dias antes do deploy em produção, uma equipe de segurança realizava testes de invasão (pentest). O resultado eram atrasos massivos ou entregas inseguras. A abordagem **Shift-Left** move os testes e validações de segurança para o início ("à esquerda") da linha do tempo: no editor do programador, no commit e nos primeiros passos da esteira automatizada.
- **Por que importa:** Corrigir uma falha enquanto o desenvolvedor ainda está escrevendo o código custa até **60 vezes menos** do que corrigir uma vulnerabilidade explorada em produção com vazamento de dados na imprensa.

DevSecOps (Development, Security, and Operations)

- **O que é:** Evolução da cultura DevOps que integra a segurança como um **requisito não-funcional de primeira classe** em todas as etapas de entrega de software. Não se trata de uma ferramenta única, mas da união de pessoas, processos automatizados e ferramentas que garantem entregas rápidas, previsíveis e blindadas.

- **No Clube Aurora:** Representado pela esteira de CI/CD que analisa desde a sintaxe do Dockerfile até a segurança dinâmica da aplicação em tempo de execução.

---

2. A Tríade CIA no Contexto do Negócio

A Tríade CIA é o alicerce fundamental de qualquer política de segurança da informação mundial:

1. Confidencialidade (Confidentiality)

- **Definição:** Garantir que a informação seja acessível estritamente por pessoas, processos ou sistemas autorizados.
- **Impacto no Clube Aurora:** O clube atende indivíduos de patrimônio ultra-alto (UHNWI - Ultra High Net Worth Individuals). Dados como endereços residenciais de celebridades, horários de voos em jatos particulares e faturas de cartão de crédito não podem ser lidos por funcionários não autorizados nem expostos na internet. A quebra de confidencialidade destrói o modelo de negócios por completo.

2. Integridade (Integrity)

- **Definição:** Assegurar que os dados, parâmetros e códigos permaneçam exatos, completos e protegidos contra adulteração ou corrupção não autorizada.
- **Impacto no Clube Aurora:** Se um invasor puder alterar o valor de uma reserva de jato privado de R$ 50.000 para R$ 1,00, ou se puder mudar o destino de um voo no banco de dados, o serviço entra em colapso logístico e financeiro.

3. Disponibilidade (Availability)

- **Definição:** Assegurar que os sistemas, bancos de dados e serviços estejam operacionais e acessíveis para os usuários legítimos sempre que necessário.
- **Impacto no Clube Aurora:** Se a aplicação cair durante o fechamento de um lote exclusivo em um leilão de arte, membros VIP não conseguirão efetuar seus lances, gerando perda imediata de faturamento e graves danos à reputação da marca.

---

3. Modelagem de Ameaças: Framework STRIDE Esmiuçado

Desenvolvido originalmente pela Microsoft, o framework **STRIDE** categoriza as ameaças de segurança que podem afetar um software. Cada letra representa uma categoria de ataque com um objetivo específico do invasor:

Letra	Categoria de Ameaça	Objetivo do Atacante	O que quebra?	Exemplo Real no Clube Aurora
S	**Spoofing** (Falsificação)	Fingir ser quem não é (forjar identidade de outro usuário, serviço ou cliente).	Autenticação	Um atacante forjar a sessão da gerente Carla para emitir ordens de serviço.
T	**Tampering** (Adulteração)	Modificar dados em trânsito ou no banco de dados sem autorização.	Integridade	Injetar comandos SQL maliciosos para alterar o status de reservas no SQLite.

Letra	Categoria de Ameaça	Objetivo do Atacante	O que quebra?	Exemplo Real no Clube Aurora
R	**Repudiation** (Repúdio)	Realizar uma ação danosa e negar a autoria por falta de registros e provas.	Auditoria / Não-Repúdio	Um funcionário desonesto deletar o histórico de um cliente VIP e o sistema não registrar logs de quem fez isso.
I	**Information Disclosure** (Vazamento)	Obter acesso a dados confidenciais ou segredos internos do sistema.	Confidencialidade	Chaves de API e senhas expostas no arquivo `.env` dentro do histórico do Git.
D	**Denial of Service** (Negação de Serviço)	Esgotar recursos da máquina ou da aplicação para torná-la inacessível.	Disponibilidade	Executar consultas que travam o banco de dados ou derrubam o contêiner por consumo de memória.
E	**Elevation of Privilege** (Elevação de Privilégio)	Um usuário de menor privilégio obter poderes administrativos indevidos.	Autorização	Um cliente ou operador comum conseguir acessar a rota `/admin/users` e alterar seu próprio papel para `admin`.

---

4. Governança, Identidade e Controle de Acesso (IAM/RBAC)

Princípio do Menor Privilégio (PoLP - Principle of Least Privilege)

- **O que é:** Regra de ouro da segurança segundo a qual todo usuário, processo ou software deve operar utilizando estritamente as permissões mínimas indispensáveis para cumprir sua tarefa específica, e nada mais.
- **Aplicação Prática:** Se a gerente Carla precisa apenas visualizar e cadastrar agendamentos de clientes, ela **não deve** ter permissão para reiniciar o servidor, alterar configurações de rede ou modificar usuários do banco.

Segregação de Funções (SoD - Separation of Duties)

- **O que é:** Princípio de governança corporativa e compliance (SOX, ISO 27001) que impede a concentração excessiva de poderes críticos em uma única pessoa, reduzindo o risco de fraudes e erros acidentais.
- **No Caso Carla e Tio Armando:** Carla solicitou permissões de superadministradora ("admin total") para "agilizar o atendimento". A concessão desse acesso violaria frontalmente a SoD: uma profissional da área de negócios passaria a ter poderes de infraestrutura técnica sem supervisão de TI. A solução correta é criar um papel sob medida (`concierge_manager`) com acesso pleno às telas de atendimento, sem poderes administrativos no sistema.

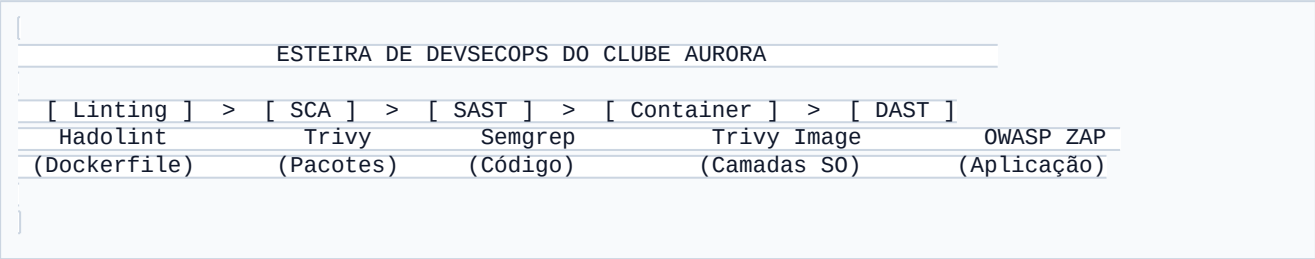
RBAC (Role-Based Access Control - Controle de Acesso Baseado em Papéis)

- **O que é:** Modelo de autorização onde os acessos aos recursos do sistema não são atribuídos diretamente aos nomes das pessoas, mas sim a **papéis** (funções ou cargos). Cada papel possui um conjunto pré-determinado de permissões.
- **Estrutura Típica:**
- `admin`: Manutenção técnica, gerenciamento de papéis e auditoria do sistema.
- `staff` / `concierge_manager`: Cadastro, consulta e gestão de experiências de clientes VIP.
- `client`: Visualização estrita de suas próprias reservas e perfil.

---

5. Tipos de Testes Automatizados na Esteira DevSecOps

Uma esteira madura de DevSecOps implementa a estratégia de **Defesa em Profundidade**, combinando múltiplos scanners especializados:



1. Linter de Infraestrutura (Hadolint)

- **Como atua:** Analisa arquivos de receita de contêiner (`Dockerfile`) verificando conformidade com as melhores práticas de mercado (regras DLxxxx).
- **O que detecta:** Uso de tags mutáveis (`:latest`), ausência de limpeza de cache de pacotes, instruções desnecessárias.

2. SCA (Software Composition Analysis - Trivy Filesystem)

- **Como atua:** Lê os arquivos de manifesto de dependências (como `requirements.txt` no Python ou `package.json` no Node.js) e cruza as versões instaladas com bases mundiais de vulnerabilidades públicas (CVEs).
- **O que detecta:** Bibliotecas de terceiros antigas com brechas conhecidas (ex.: CVEs de corrupção de memória ou negação de serviço no `urllib3` e `werkzeug`).

3. SAST (Static Application Security Testing - Semgrep)

- **Como atua:** Realiza uma inspeção estática no código-fonte sem executá-lo, como um "raio-X" da lógica de programação.
- **O que detecta:** Falhas de codificação do time de desenvolvimento, como injeção de comandos SQL, uso de algoritmos criptográficos fracos e validações ausentes.

4. Varredura de Imagem Contêiner (Trivy Image)

- **Como atua:** Descompacta as camadas da imagem Docker gerada e inspeciona tanto as bibliotecas da linguagem quanto os binários do sistema operacional base (pacotes Debian/Alpine como `openssl` , `glibc` , `curl`).
- **O que detecta:** Vulnerabilidades no sistema operacional subjacente que poderiam permitir a um invasor obter controle do contêiner.

5. DAST (Dynamic Application Security Testing - OWASP ZAP)

- **Como atua:** Teste de "caixa-preta" que simula um atacante real disparando requisições HTTP maliciosas contra a aplicação web enquanto ela está rodando.
- **O que detecta:** Cabeçalhos de segurança ausentes, cookies desprotegidos, falhas de redirecionamento aberto e brechas que só aparecem em tempo de execução.

---

6. Anatomia das Vulnerabilidades do Clube Aurora (F1 a F7)

F1: Exposição de Segredos no Git (.env rastreado)

- **O Risco:** Armazenar chaves secretas de produção (`SECRET_KEY` , senhas de banco) em arquivos de texto e comitá-los no histórico do Git. Mesmo que o repositório seja privado, colaboradores, forks ou invasores com acesso ao histórico conseguem ler as credenciais.
- **A Solução:** Adicionar o arquivo `.env` ao `.gitignore` , desindexá-lo do Git (`git rm --cached .env`) e disponibilizar um modelo seguro (`.env.example`) apenas com variáveis vazias.

F2: Dependências Vulneráveis com CVEs (SCA)

- **O Risco:** O arquivo `requirements.txt` declarava versões antigas do Flask (`2.2.5`), Werkzeug (`2.2.3`) e urllib3 (`1.26.15`), acumulando vulnerabilidades críticas como negação de serviço e bypass de autenticação.
- **A Solução:** Atualizar as bibliotecas para versões estáveis e recomendadas (`flask>=3.0.3` , `urllib3>=2.8.0` , etc.) e fixar as versões na esteira.

F3: Injeção de SQL (SQL Injection - SQLi)

- **O Risco:** Construir instruções SQL concatenando strings fornecidas pelo usuário através de f-strings: `f"SELECT * FROM clients WHERE full_name LIKE '%{name}%'"`
  Se o usuário digitar `' OR '1'='1` , a consulta se torna verdadeira para todos os registros e vaza a tabela inteira de clientes VIP.
- **A Solução:** Utilizar **Consultas Parametrizadas** (Prepared Statements) com o placeholder seguro `?` . O banco de dados trata a entrada do usuário estritamente como dado literal, impedindo que comandos maliciosos alterem a estrutura da consulta SQL:
  `db.execute("SELECT * FROM clients WHERE full_name LIKE ? ORDER BY id", (f"%{name}%",))`

F4: Dockerfile sem Hardening

- **O Risco:** Utilizar a imagem base mutável `python:latest` e executar o processo principal da aplicação como o superusuário `root` do Linux dentro do contêiner. Em caso de brecha na aplicação, o atacante obtém poderes de root dentro do contêiner, facilitando o escape para a máquina hospedeira.
- **A Solução:** Fixar a versão específica da imagem base (`python:3.11-slim`), criar um usuário sem privilégios (`USER nonroot`), configurar multi-stage build e adicionar monitoramento de integridade (`HEALTHCHECK`).

F5: Ausência de Security Headers HTTP

- **O Risco:** A aplicação respondia a requisições HTTP sem cabeçalhos de segurança essenciais, permitindo ataques de sequestro de clique (Clickjacking), inclusão de scripts de terceiros (XSS) e interpretação incorreta de tipos de arquivos (MIME sniffing).
- **A Solução:** Injetar os cabeçalhos defensivos recomendados pela OWASP:

- `Content-Security-Policy: default-src 'self'` (impede execução de scripts externos).
- `X-Frame-Options: DENY` (impede que a página seja aberta dentro de um iframe falso).
- `X-Content-Type-Options: nosniff` (força o navegador a respeitar o tipo MIME).
- `Strict-Transport-Security` (HSTS - obriga conexões seguras HTTPS).

F6: Quebra de Menor Privilégio e Elevação de Papel

- **O Risco:** Novos usuários eram criados automaticamente com o papel `admin` por padrão, e a rota de alteração de papéis `/admin/users` não validava se o solicitante possuía privilégios administrativos.
- **A Solução:** Definir o papel padrão como `client` ou `staff` e travar as rotas administrativas com o decorador `@auth.require_role("admin")`, retornando HTTP 403 Forbidden para acessos indevidos.

F7: Infraestrutura como Código Permissiva (docker-compose)

- **O Risco:** O arquivo `docker-compose.yml` declarava a flag `privileged: true` e montava o socket interno do Docker do hospedeiro (`/var/run/docker.sock`). Isso dava ao contêiner controle irrestrito sobre todo o ambiente de virtualização.
- **A Solução:** Remover o modo privilegiado, desvincular o socket do Docker e operar com permissões mínimas de rede.

---

7. Hardening e Segurança em Contêineres (Docker)

Por que a tag `:latest` é um perigo em produção?

A tag `:latest` é mutável e flutuante. A cada nova compilação, ela pode baixar uma versão completamente diferente da linguagem ou do sistema operacional, introduzindo incompatibilidades silenciosas ou novas vulnerabilidades desconhecidas. Em ambientes de missão crítica, a imagem base deve ser sempre **imutável e fixada** (ex.: `python:3.11-slim`).

Por que NUNCA rodar contêineres como `root`?

Por padrão, o usuário dentro de um contêiner Docker possui UID 0 (root). Embora o contêiner isole processos via namespaces e cgroups, vulnerabilidades no kernel do Linux ou má configurações (como volumes mal mapeados) podem permitir um ataque de **Container Escape**. Se o atacante escapar de um processo que roda como root, ele se torna root do servidor hospedeiro. Executar com usuário sem privilégios (`USER nonroot` ou `USER 10001`) mitiga quase totalmente esse risco.

Compilação Multi-Stage (Multi-Stage Build)

Técnica onde o `Dockerfile` utiliza múltiplos estágios (`FROM ... AS builder` e `FROM ... AS runner`). O primeiro estágio contém compiladores e pacotes de desenvolvimento pesados para gerar os binários; o segundo estágio copia apenas os artefatos finais para uma imagem limpa e enxuta, reduzindo drasticamente a superfície de ataque e o tamanho do contêiner.

---

8. Auditoria, Observabilidade e Não-Repúdio

O Princípio do Não-Repúdio (Non-Repudiation)

Garantia técnica de que uma entidade que realizou determinada transação ou modificação no sistema não consiga negar a autoria de sua ação. Isso é obtido através de registros de auditoria imutáveis com registros de

data/hora (timestamp), identificador do usuário e endereço IP.

O que uma Tabela de Auditoria (`audit_log`) Deve Registrar?

Para atender à LGPD e às normas de conformidade bancária/concierge, o sistema deve registrar:
1. **Quem:** Identificador do usuário que realizou a ação (`user_id`).
2. **Quando:** Timestamp exato em fuso horário universal (`timestamp UTC`).
3. **De Onde:** Endereço IP e agente de navegação do solicitante (`ip_address`).
4. **O Quê:** Ação executada (ex.: `CREATE_USER`, `DELETE_BOOKING`, `ACCESS_VIP_DATA`).
5. **Resultado:** Se a operação foi autorizada (`SUCCESS`) ou bloqueada (`DENIED/FAILED`).

---

9. Gestão de Riscos, Exceções e Métricas Corporativas

CVE (Common Vulnerabilities and Exposures)

Identificador internacional padronizado atribuído a vulnerabilidades públicas (ex.: `CVE-2024-1135`). Permite que times de segurança e ferramentas no mundo inteiro falem a mesma língua sobre uma falha específica.

CVSS (Common Vulnerability Scoring System)

Métrica numérica que vai de `0.0` a `10.0` utilizada para classificar a severidade de uma vulnerabilidade:
* **0.1 a 3.9:** Baixa (Low)
* **4.0 a 6.9:** Média (Medium)
* **7.0 a 8.9:** Alta (High)
* **9.0 a 10.0:** Crítica (Critical)

Falso Positivo (False Positive)

Alerta emitido por uma ferramenta de segurança que sinaliza incorretamente um trecho de código ou biblioteca como vulnerável, quando na verdade, pelo contexto ou arquitetura do sistema, o risco não existe ou a função vulnerável não é invocada.

Aceite Formal de Risco e `.trivyignore`

Quando uma biblioteca apresenta um CVE conhecido, mas a falha não tem impacto prático na aplicação e o mantenedor ainda não lançou uma versão corrigida, a governança de segurança permite um **Aceite Formal de Risco**. No Trivy, isso é registrado no arquivo `.trivyignore`, que deve conter obrigatoriamente: o CVE aceito, a justificativa técnica detalhada e uma data limite de expiração para revisão.